

The Method of Gaussian Quadrature

Katie Smela

MA418

Department of Mathematics and Computer Science
Colorado College
April 17, 2026

Contents

1	Introduction and Statement of Problem	2
2	Background	2
2.1	General Idea of Quadrature	3
2.2	Newton-Cotes Rules	3
2.2.1	Trapezoid Rule	3
2.2.2	Simpson's Rule	3
2.3	Motivation for Gaussian Quadrature	4
3	Derivation of Gaussian Quadrature	4
3.1	Exactness and the Method of Undetermined Coefficients	4
3.2	Orthogonal Polynomials	5
3.3	Choice of Nodes	5
3.4	Exactness of Polynomials	5
3.4.1	Integral Analysis	6
3.4.2	Quadrature Rule Analysis	6
3.4.3	Exactness on the Remainder	7
3.5	Choice of Weights	7
3.6	Approximating Over the General Interval $[a, b]$	8
4	Application of Method to Example	8
5	Convergence and When Gaussian Quadrature Works	10
6	Numerical Experiments	10
7	Conclusion	22

1 Introduction and Statement of Problem

Numerical integration is a foundational problem in applied mathematics, finding importance in fields ranging from physics and engineering to probability and data science. In many cases, one seeks to evaluate definite integrals of the form

$$\int_a^b f(x)dx.$$

However, it is often the case that an exact anti-derivative for $f(x)$ is unknown, or the function itself is only known through discrete data and expensive computations. In such cases, quadrature methods provide efficient and practical approximations.

A general quadrature rule approximates the integral using a weighted sum of function values:

$$Q_n(f) = \sum_{i=1}^n w_i f(x_i)$$

where the nodes x_i and weights w_i are chosen to achieve a certain level of accuracy. A central question in the design of quadrature methods is how to select these nodes and weights to maximize the accuracy of the approximation for a given number of function evaluations.

Classical methods, such as the trapezoid rule or Simpson's rule, use evenly spaced nodes and achieve varying degrees of accuracy depending on the number of points used in the approximation. However, these methods are not optimal. For a fixed number of nodes, it is possible to create quadrature methods that significantly increase the accuracy of the integral approximation.

Gaussian quadrature provides such an optimal construction. By allowing both the nodes and weights to vary, and by choosing the nodes to be the roots of orthogonal polynomials, specifically Legendre polynomials on the interval $[-1, 1]$, Gaussian quadrature achieves the highest possible degree of precision. More specifically, an n -point Gaussian quadrature scheme integrates all polynomials of degree up to $2n - 1$ exactly. [3]

The purpose of this paper is to develop the theory behind Gaussian quadrature, explain how it achieves the stated optimal accuracy, and investigate its performance through numerical simulations. I derive the method using orthogonal polynomials, analyze its properties, and compare its behavior with other quadrature rules.

2 Background

Before developing Gaussian quadrature, it is useful to review standard numerical integration techniques and their limitations to provide context and motivation for the construction of more advanced quadrature rules.

2.1 General Idea of Quadrature

As previously mentioned, quadrature methods seek to estimate definite integrals of the form

$$\int_a^b f(x)dx$$

by evaluating the function f at a finite set of points and forming a weighted sum:

$$Q_n(f) = \sum_{i=1}^n w_i f(x_i) \quad (1)$$

The accuracy of quadrature rules is measured by their degrees of precision, defined as the largest integer m such that the rule integrates polynomials of degree at most m exactly.

2.2 Newton-Cotes Rules

A widely used class of quadrature rules is given by the Newton-Cotes formulas that draw an interpolating polynomial through equally spaced nodes and then integrate [3].

2.2.1 Trapezoid Rule

The trapezoid rule uses evaluations of f at two points x_0 and x_1 , and approximates the integrand with a linear interpolant. This method has a degree of precision 1, meaning that it is exact for all linear polynomials.

$$\int_{x_0}^{x_1} f(x)dx \approx \frac{h}{2} [f(x_0) + f(x_1)]$$

where $h = x_1 - x_0$.

2.2.2 Simpson's Rule

Simpson's rule uses three equally spaced nodes and a quadratic interpolant.

$$\int_{x_0}^{x_2} f(x)dx \approx \frac{h}{3} [f(x_0) + 4f(x_1) + f(x_2)]$$

where $h = \frac{1}{2}(x_2 - x_0)$. This rule has a degree of precision 3, meaning that it is exact for all polynomials up to cubic degree.

2.3 Motivation for Gaussian Quadrature

While Newton-Cotes methods are relatively simple and widely used, they suffer from a fundamental limitation: the nodes x_i are fixed in advance, leaving only the weights w_i to be strategically chosen for optimal accuracy. As a result, Newton-Cotes quadratures generally achieve a degree of precision no greater than $n + 1$ [2]. Increasing the number of nodes used in a quadrature method often increases the accuracy of the estimation, but at the cost of a greater number of function evaluations and potentially introducing more numerical instability with higher degree polynomials. This raises a natural question: can we instead do better by choosing both the weights and nodes optimally?

Gaussian quadrature answers this question affirmatively. By allowing nodes to vary, it is possible to construct quadrature rules that achieve the maximum possible degree of precision. In particular, an n -point Gaussian quadrature rule can be constructed to integrate all polynomials of degree $2n - 1$ exactly, nearly doubling the degree of precision of classic methods that fix nodes [3].

The key to this construction lies in the use of orthogonal polynomials, which provide a natural framework for optimally selecting weights and nodes. In the following sections, I develop this theory and show it leads to the Gaussian quadrature method.

3 Derivation of Gaussian Quadrature

We seek to choose nodes x_i and weights w_i such that

$$Q_n(p) = \int_a^b p(x) dx$$

for all polynomials of at most degree m , given some fixed n number of nodes.

3.1 Exactness and the Method of Undetermined Coefficients

To construct such a rule, we begin by requiring that the rule be exact for the basis monomials [1]

$$1, x, x^2, x^3, \dots, x^m.$$

Since any polynomial of degree m can be written as a linear combination of basis elements, if we can enforce exactness on the basis, we can thus enforce exactness on all polynomials up to degree m .

This leads to a system of equations

$$\sum_{i=1}^n w_i x_i^k = \int_a^b x^k dx \quad \text{for } k = 0, 1, 2, \dots, m.$$

This quadrature has $2n$ unknown values, n weights and n nodes, and $m + 1$ conditions to satisfy. Thus, the maximal possible degree of precision is $m = 2n - 1$. While the system above characterizes the optimal quadrature rule, solving this non-linear system directly is difficult, especially for large n . This motivates a different, more inspired selection of weights and nodes.

3.2 Orthogonal Polynomials

Before describing a substitute approach to the method of undetermined coefficients, a brief aside into orthogonal polynomials is necessary to provide context for the structure of Gaussian quadrature.

The set of non-zero functions $\{p_0, p_1, \dots, p_n\}$ on the interval $[a, b]$ is said to be orthogonal if

$$\int_a^b p_j(x)p_k(x)dx = 0 \text{ for } j \neq k \quad [3].$$

Note that the Legendre polynomials, given by the form

$$p_i(x) = \frac{1}{p^i i!} \frac{d^i}{dx^i} [(x^2 - 1)^i]$$

make up such a set on $[-1, 1]$ [3]. In particular, the polynomial $p_n(x)$ of degree n satisfies

$$\int_{-1}^1 p_n(x)q(x)dx = 0$$

for all polynomials $q(x)$ of degree less than n [3]. This fact will become useful later in the derivation.

3.3 Choice of Nodes

Let $p_n(x)$ denote the Legendre polynomial of degree n and let $x_1, x_2, \dots, x_n$ be its n distinct roots. These roots lie in the interval $(-1, 1)$ and will be used as the nodes for the quadrature. This choice is motivated by the orthogonality property: by selecting the nodes as the roots of $p_n(x)$, we ensure that the polynomial $p_n(x)$ vanishes at all quadrature points.

3.4 Exactness of Polynomials

I will now show that this specific selection of n nodes produces a quadrature scheme that is exact for polynomials of degree up to $2n - 1$ on the interval $[-1, 1]$. Let $P(x)$ be any polynomial of degree at most $2n - 1$. By polynomial division, we can write

$$P(x) = P_n(x)Q(x) + R(x)$$

where $\deg(Q) \leq n - 1$ and $\deg(R) < n$, and $P_n(x)$ is a Legendre polynomial of degree n . We seek to show

$$Q_n(P) = \int_{-1}^1 P(x)dx = I(P),$$

and to do so, we will analyze the quadrature rule and integral separately and then show equivalence.

3.4.1 Integral Analysis

Applying the linearity property of integrals, we can write

$$I(P) = \int_{-1}^1 P(x)dx = \int_{-1}^1 P_n(x)Q(x)dx + \int_{-1}^1 R(x)dx.$$

Because $Q(x)$ has degree at most $n - 1$, orthogonality implies

$$\int_{-1}^1 P_n(x)Q(x)dx = 0.$$

Thus

$$I(P) = \int_{-1}^1 P(x)dx = \int_{-1}^1 R(x)dx.$$

3.4.2 Quadrature Rule Analysis

Evaluating the quadrature rule, we have

$$\begin{aligned} Q_n(f) &= \sum_{i=1}^n w_i P(x_i) \\ &= \sum_{i=1}^n w_i [P_n(x_i)Q(x_i) + R(x_i)]. \end{aligned}$$

Because each x_i is a root of the polynomial $P_n(x)$, $P_n(x_i)Q(x_i) = 0$ for each i . Hence

$$Q_n(f) = \sum_{i=1}^n w_i R(x_i).$$

3.4.3 Exactness on the Remainder

Our problem has been reduced to showing the quadrature is exact on the remainder polynomial $R(x)$. Note that $R(x)$ has degree less than n and is uniquely determined by its values at the nodes $x_1, x_2, \dots, x_n$. It follows that $R(x)$ coincides with its interpolating polynomial of degree at most $n - 1$. By construction, the weights w_i are chosen so that the quadrature rule integrates all polynomials of at most degree $n - 1$ exactly. Therefore,

$$\sum_{i=1}^n w_i R(x_i) = \int_{-1}^1 R(x) dx.$$

Combining the above results then yields

$$Q_n(P) = \int_{-1}^1 P(x) dx$$

and the constructed quadrature method integrates polynomials of at most degree $2n - 1$, the maximum degree of precision for an n -point quadrature scheme, exactly.

3.5 Choice of Weights

While the logic behind the argument of the degree of precision of Gaussian quadrature relied heavily on the properties of the selected nodes, we should also make note of the method used to select weights after the nodes are determined.

The key idea with quadrature in general is to approximate some integrand $f(x)$ with an interpolating polynomial of degree at most $n - 1$ through the nodes $x_1, x_2, \dots, x_n$. We will proceed with the Lagrange form of an interpolating polynomial given by

$$L(x) = \sum_{i=1}^n f(x_i) \varphi_i(x)$$

where the Lagrange basis polynomials $\varphi_i(x)$ are defined as

$$\prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

We then approximate the integral of $f(x)$ by integrating the interpolating polynomial.

$$\int_{-1}^1 f(x) dx \approx \int_{-1}^1 L(x) dx$$

Using the linearity of integrals, this becomes

$$\begin{aligned}\int_{-1}^1 L(x)dx &= \int_{-1}^1 \sum_{i=1}^n f(x_i)\varphi_i(x)dx \\ &= \sum_{i=1}^n \int_{-1}^1 f(x_i)\varphi_i(x)dx \\ &= \sum_{i=1}^n f(x_i) \int_{-1}^1 \varphi_i(x)dx\end{aligned}$$

This motivates defining the weights w_i as follows:

$$w_i = \int_{-1}^1 \varphi_i(x)dx$$

3.6 Approximating Over the General Interval $[a, b]$

Recall that this quadrature method relied heavily on the orthogonality of the Legendre polynomials over the interval $[-1, 1]$, and hence our subsequent derivations have integrated over the bounds $a = -1$ to $b = 1$. To approximate integrals of the form

$$\int_a^b f(x)dx,$$

the problem must be translated back to $[-1, 1]$ where the orthogonality property is valid. Using the substitution $t = (2x - a - b)/(b - a)$, we find that

$$\int_a^b f(x)dx = \int_{-1}^1 f\left(\frac{(b-a)t + b + a}{2}\right) \frac{b-a}{2} dt \quad [3].$$

4 Application of Method to Example

We will demonstrate the Gaussian Quadrature algorithm by working through an example.

Let us approximate the definite integral

$$\int_{-1}^1 e^{\frac{-x^2}{2}} dx.$$

using a two-point Gaussian quadrature. The first step is to determine the nodes through which we will interpolate the integrand. Recall that the nodes for a two-point method are found by solving for the roots of the second-degree Legendre polynomial:

$$P_2(x) = \frac{1}{8} \frac{d^2}{dx^2} [(x^2 - 1)^2]$$

Let $g(x) = (x^2 - 1)^2$ and differentiate twice.

$$\begin{aligned}g(x) &= x^4 - 2x^2 + 1 \\g'(x) &= 4x^3 - 4x \\g''(x) &= 12x^2 - 4 = 4(3x^2 - 1)\end{aligned}$$

Hence we have $P_2(x) = \frac{1}{2}(3x^2 - 1)$. Then we set $P_2(x) = 0$ to solve for the roots.

$$\begin{aligned}\frac{1}{2}(3x^2 - 1) &= 0 \\3x^2 &= 1 \\x &= \pm \frac{\sqrt{3}}{3}\end{aligned}$$

Thus, we have the beginning of our quadrature scheme:

$$Q_n(f) = \sum_{i=1}^2 w_i f(x_i) = w_1 f\left(\frac{\sqrt{3}}{3}\right) + w_2 f\left(\frac{-\sqrt{3}}{3}\right)$$

Then we must solve for the weights w_1 and w_2 using the formula we derived in the previous section:

$$w_i = \int_{-1}^1 \varphi_i(x) dx$$

Hence

$$\begin{aligned}w_1 &= \int_{-1}^1 \varphi_1(x) dx \\&= \int_{-1}^1 \frac{x - x_2}{x_1 - x_2} dx \\&= \frac{1}{x_1 - x_2} [x^2 - x_2 x]_{x=-1}^{x=1} \\&= \frac{-2x_2}{x_1 - x_2} \\&= \frac{-2\left(\frac{-\sqrt{3}}{3}\right)}{2\frac{\sqrt{3}}{3}} \\&= 1\end{aligned}$$

A similar derivation shows that w_2 also equals one. Hence, we achieve our approximation:

$$\int_{-1}^1 e^{\frac{-x^2}{2}} dx \approx e^{\frac{-(\frac{\sqrt{3}}{3})^2}{2}} + e^{\frac{-(-\frac{\sqrt{3}}{3})^2}{2}} \approx 1.69296$$

5 Convergence and When Gaussian Quadrature Works

Gaussian quadrature achieves the highest possible degree of precision for a quadrature rule, integrating polynomials of up to degree $2n - 1$ exactly. This optimality leads to excellent convergence properties when applied to sufficiently smooth functions.

For a general function $f(x) \in C^{(2n)}[a, b]$, the error of Gaussian quadrature can be expressed in the form

$$\int_a^b f(x)dx - Q_n(f) = \frac{f^{(2n)}(\xi)}{(2n)!}(p_n, p_n)$$

for some $\xi \in (a, b)$, where we define $(p_n, p_n) = \int_a^b p_n(x)p_n(x)dx$ and p_n is the degree n Legendre polynomial [4]. This expression shows that the error is dependent on the $2n$ th derivative of f , in contrast to other methods like Simpson's rule, where the error depends on a fixed, low-order derivative and the fixed step size h . As a result, Gaussian quadrature exhibits rapid convergence for smooth enough functions. In particular, for sufficiently differentiable functions, the error decreases quickly as n increases.

While Gaussian quadrature is highly effective for smooth functions, its performance can degrade in certain situations. Because this method relies on polynomial interpolation, its accuracy will degrade in scenarios where polynomials approximate the integrand poorly. For example, Gaussian quadrature will perform more poorly in cases where the integrand is, in general, non-smooth: $f(x)$ could have discontinuities, sharp corners, or other non-differentiability. Additionally, this method might also perform poorly for highly oscillatory functions that may require a large number of nodes to come close to capturing oscillations accurately. The scheme will also perform poorly if there is a singularity at the endpoints of the definite integral, unless a transformation is applied.

6 Numerical Experiments

In this section, I present an implementation of Gaussian quadrature in R, apply it to several test functions, and compare the results with exact integrals as we increase the number of nodes used in the interpolation. We also compare the error of Simpson's method and Gaussian quadrature as a function of the number of function evaluations required by each scheme, n evaluations by Gaussian quadrature and $n + 1$ evaluations by Simpson's method.

gaussianQuadrature

2026-04-13

Gaussian Quadrature

In this section, I implement a Gaussian quadrature scheme and apply it to several test functions to test it's performance in comparison to known exact integrals.

```
library(pracma)
```

```
## Warning: package 'pracma' was built under R version 4.5.3
```

Here I define a the function `gauss_quad()`, which takes as inputs the integrand `f`, the bounds of integration `a` and `b`, and the number of nodes `n` we wish to interpolate the integrand with.

```
gauss_quad <- function(f, a, b, n){  
  
  #compute nodes and weights  
  data <- gaussLegendre(n, a, b)  
  nodes <- data$x  
  weights <- data$w  
  
  #evaluate f at the nodes  
  fx <- sapply(nodes, f)  
  
  #get the weighted sum  
  Q <- sum(weights * fx)  
  
  return(Q)  
  
}
```

First, as a check, we confirm we achieve the same estimation for a two-point Gaussian quadrature as we did for the example worked out in section 4.

```
f <- function(x){  
  exp(-x^2/2)  
}  
  
approx_f_2pt <- gauss_quad(f, -1, 1, 2)  
approx_f_2pt
```

```
## [1] 1.692963
```

```
plotter <- function(f, a, b, real_solution, nodes){  
  n_vals <- 2:nodes  
  
  errors <- sapply(n_vals, function(n){  
    abs(real_solution - gauss_quad(f, a, b, n))  
  })  
  
  plot(n_vals, errors,  
       type = "b",  
       pch = 19,  
       xlab = "Number of Nodes (n)",  
       ylab = "Absolute Error",  
       main = "Gaussian Quadrature Error",  
       log = "y")  
}
```

Test 1: Polynomial Function

We will test our scheme on a simple polynomial function.

```
polynom <- function(x){x^5 + x^2}
```

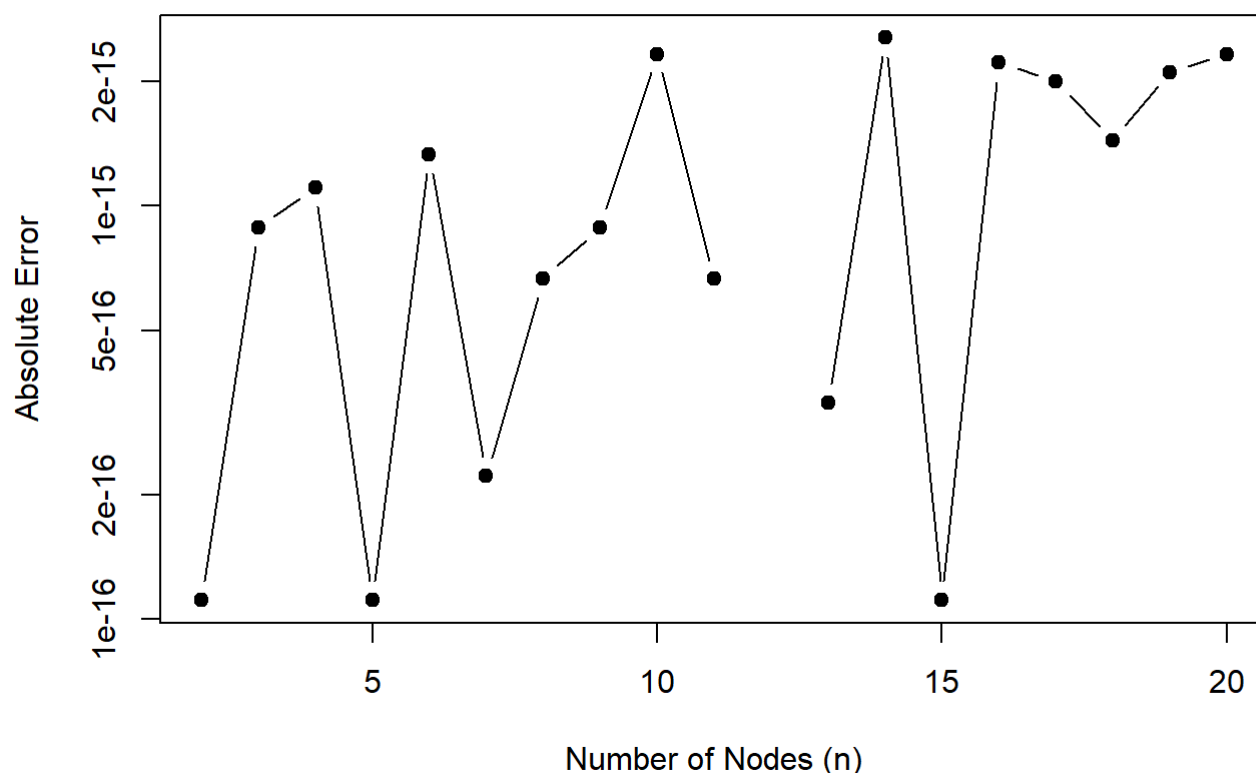
This function has a known anti-derivative and thus a known exact definite integral value of $2/3$ over the interval $[-1,1]$.

Now we will plot error as a function of n .

```
plotter(polynom, -1, 1, 2/3, 20)
```

```
## Warning in xy.coords(x, y, xlabel, ylabel, log): 1 y value <= 0 omitted from  
## logarithmic plot
```

Gaussian Quadrature Error



The oscillating behavior of the error here is depicting the result of floating-point round-off errors. Theoretically, this method should be exact for polynomials of degree no more than $2n - 1$. This test function is degree 5, so for estimations using 3 nodes or greater, the estimation should be exact.

Test 1(b): Testing polynomial with higher degree

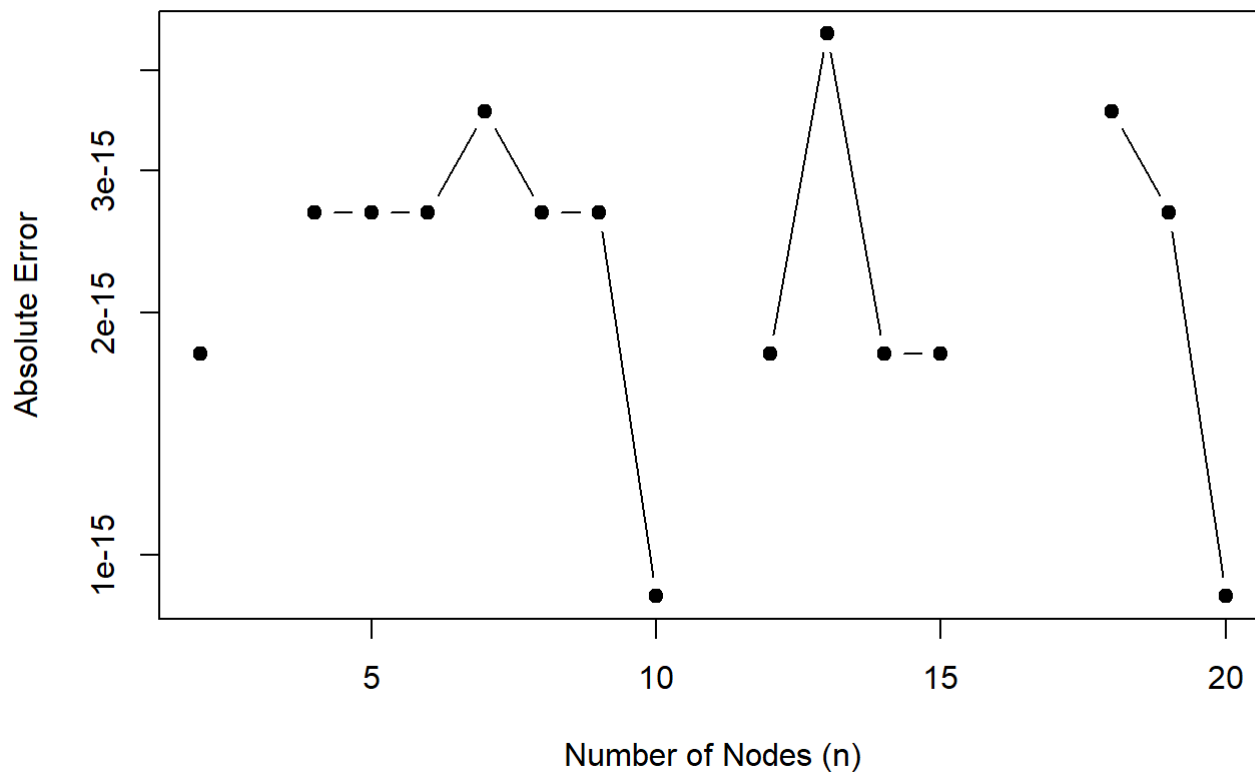
```
polynom2 <- function(x){x^9 + x^3 + 3}
```

Integrating this polynomial symbolically over the interval $[-1, 1]$ yields a value of 6. Theoretically, quadrature should be exact for estimations that use at least 5 nodes.

```
plotter(polynom2, -1, 1, 6, 20)
```

```
## Warning in xy.coords(x, y, xlabel, ylabel, log): 4 y values <= 0 omitted from
## logarithmic plot
```

Gaussian Quadrature Error



Test 2: Smooth function

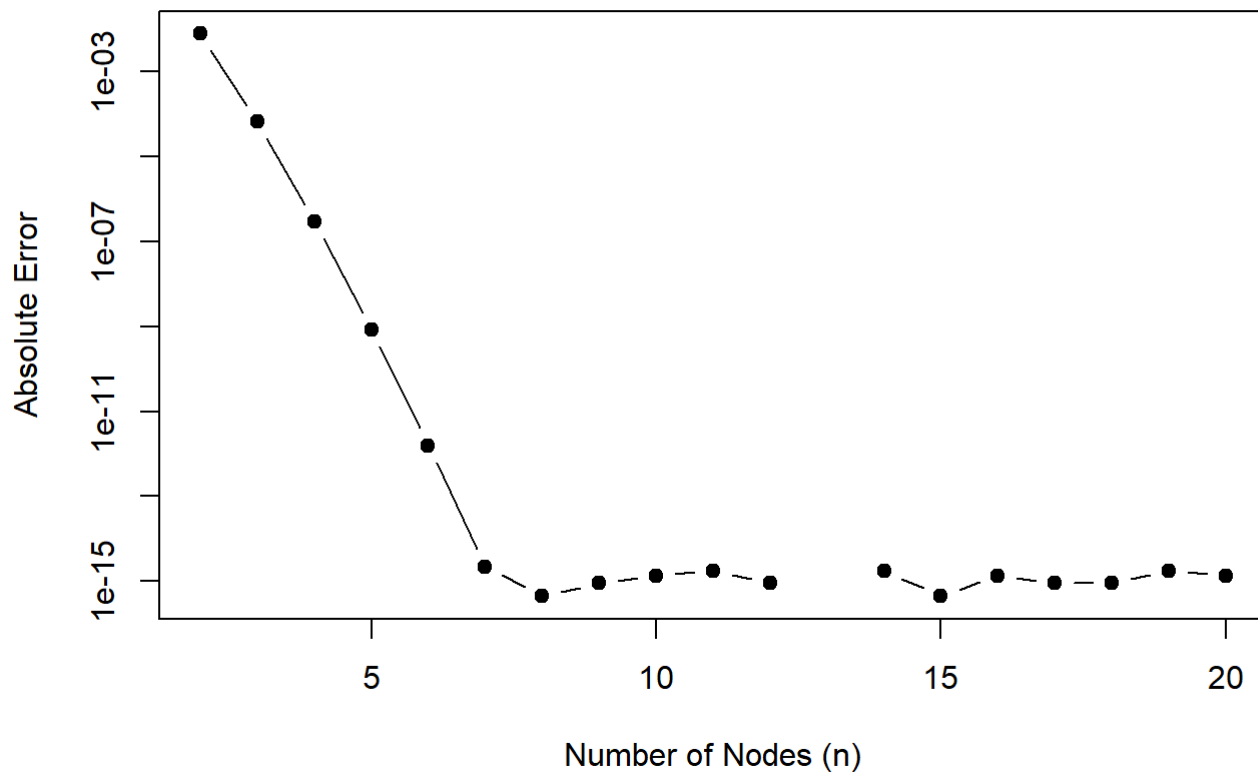
Next we test on a smooth function, $f(x) = e^x$ as it is infinitely differentiable and thus we expect our method to perform well.

Note that $\int_{-1}^1 e^x = e - \frac{1}{e} \approx 2.3504024$

```
smooth <- function(x){exp(x)}  
  
plotter(smooth, -1,1, exp(1) - 1/exp(1), 20)
```

```
## Warning in xy.coords(x, y, xlabel, ylabel, log): 1 y value <= 0 omitted from  
## logarithmic plot
```

Gaussian Quadrature Error

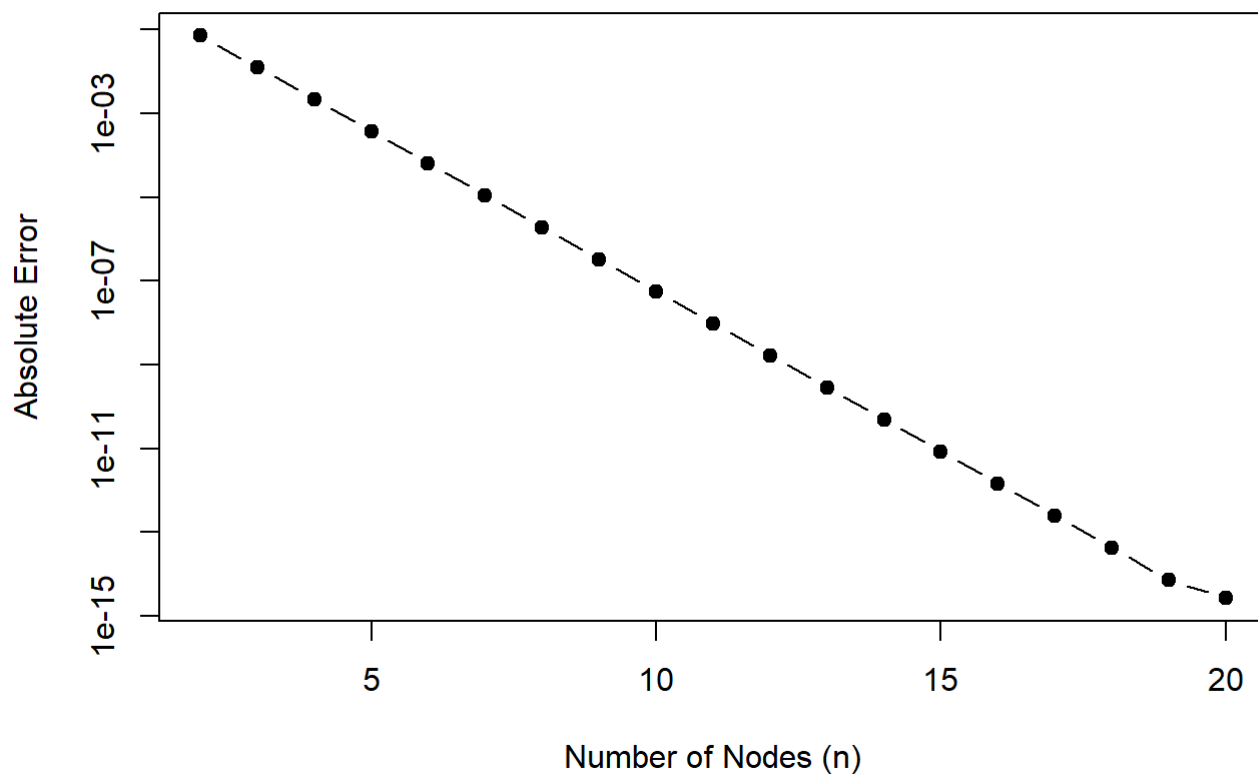


Test 3: Rational Function

Now let us test a rational function, $f(x) = \frac{1}{1+x^2}$, with exact solution on the interval $[-1, 1]$ of $\frac{\pi}{2}$.

```
rational <- function(x){1/(1+x^2)}  
  
plotter(rational, -1,1,pi/2, 20)
```

Gaussian Quadrature Error



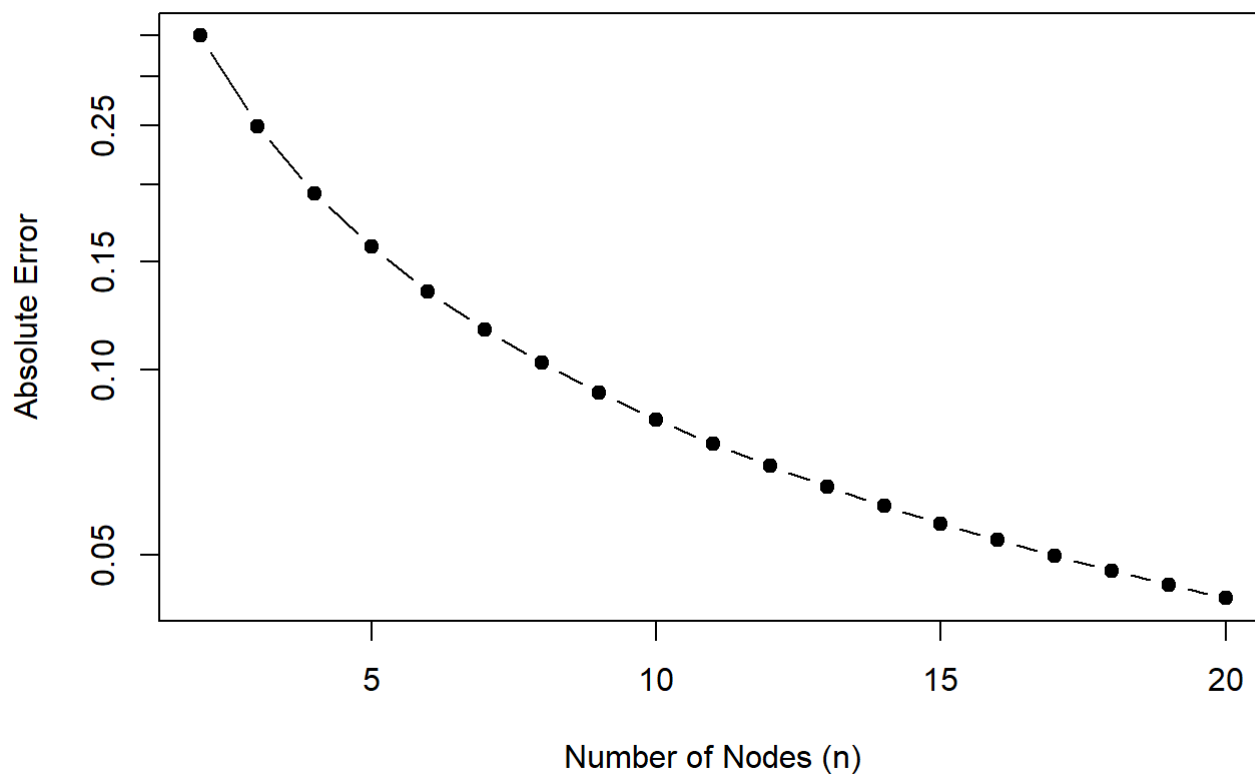
Test 4: A function changing rapidly near the bounds of integration

In accordance with section 5, we will demonstrate the accuracy of a function with a singularity at the bounds of integration, a function that is also rapidly changing near said boundary. The function $f(x) = \frac{1}{\sqrt{(x)}}$ on the interval

$[0, 1]$ has an exact solution of 2.

```
singularity <- function(x){1/sqrt(x)}  
  
plotter(singularity, 0,1,2, 20)
```

Gaussian Quadrature Error

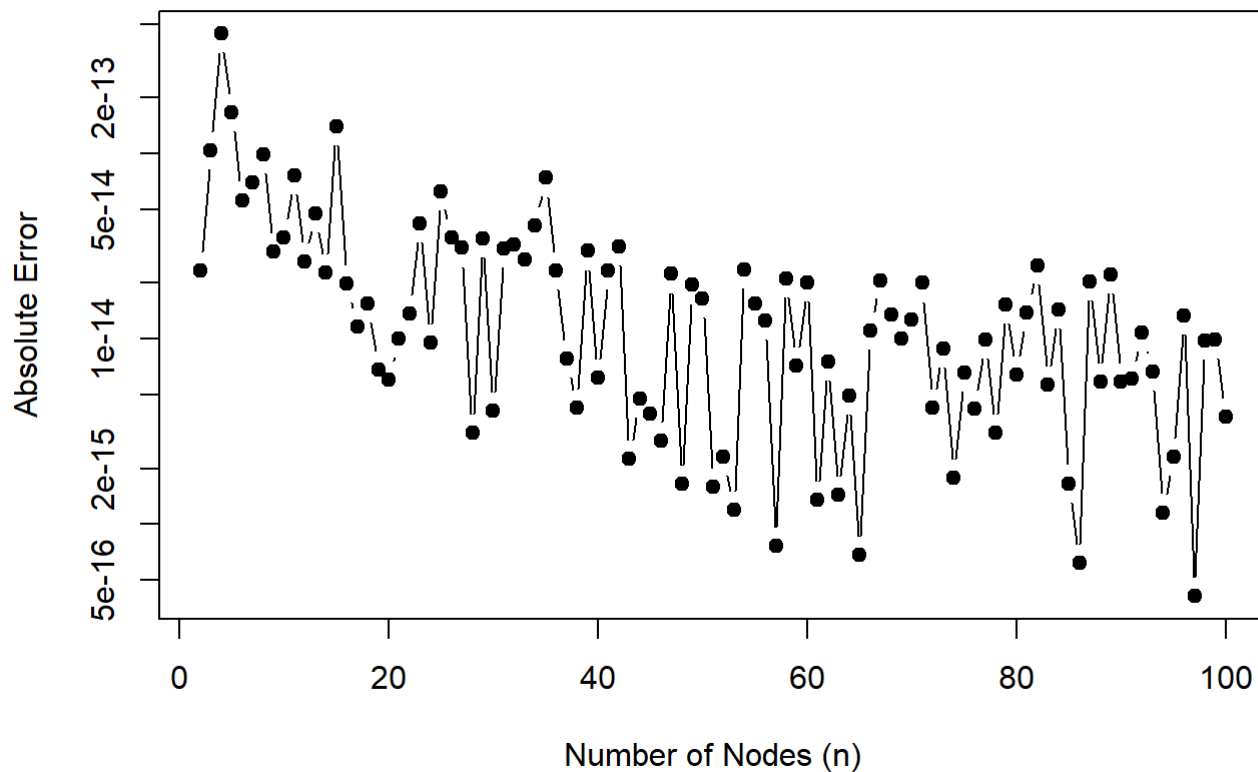


Test 5: Highly Oscillatory Functions

Finally, we will test a highly oscillatory function that, in principle, is not approximated well by low-degree polynomials. The function $f(x) = \sin(100x)$ over the interval $[0, \pi]$ integrates exactly to 0.

```
oscillatory <- function(x){sin(100*x)}  
  
plotter(oscillatory, 0, pi, 0, 100)
```

Gaussian Quadrature Error



Clearly, the number of nodes needed to gain higher levels of accuracy is extremely large with this method as compared to less oscillatory functions.

Gaussian Quadrature vs Simpson's Method:

Finally we will compare Gaussian quadrature to Simpson's rule by plotting accuracy as a function of the number of function evaluations required by the method.

```
simpsons <- function(f,a,b){
  m <- (a+b)/2

  ((b-a)/6) * (f(a) + 4*f(m) + f(b))
}
```

```
composites <- function(f, a, b, N){  
  if(N %% 2 != 0){  
    stop("N must be even")  
  }  
  
  h <- (b - a)/N # step size for a fixed number N panels  
  total <- 0  
  
  for(i in seq(0, N-2, by = 2)){  
    left <- a + i*h  
    right <- a + (i+2)*h  
    total <- total + simpsons(f, left, right)  
  }  
  
  return(total)  
}
```

To compare error between the two methods fairly, we plot error as a function of the computational cost, or the number of function evaluations in the method.

```

plotter_compare <- function(f, a, b, real_solution, max_nodes){

  n_vals <- seq(3, max_nodes, by = 2) #filtering out n to only be odd to work with simpson's

  # Gaussian errors (n evaluations)
  gauss_errors <- sapply(n_vals, function(n){
    abs(real_solution - gauss_quad(f, a, b, n))
  })

  # Simpson errors (match evaluations)
  simpson_errors <- sapply(n_vals, function(n){
    N <- n-1 # ensures N+1 ≈ n evaluations
    approx <- composites(f, a, b, N)
    abs(real_solution - approx)
  })

  # Plot Gaussian
  plot(n_vals, gauss_errors,
       type = "b",
       pch = 19,
       log = "y",
       xlab = "Number of Function Evaluations",
       ylab = "Absolute Error",
       main = "Gaussian vs Simpson Convergence")

  # Add Simpson
  lines(n_vals, simpson_errors,
       type = "b",
       pch = 17)

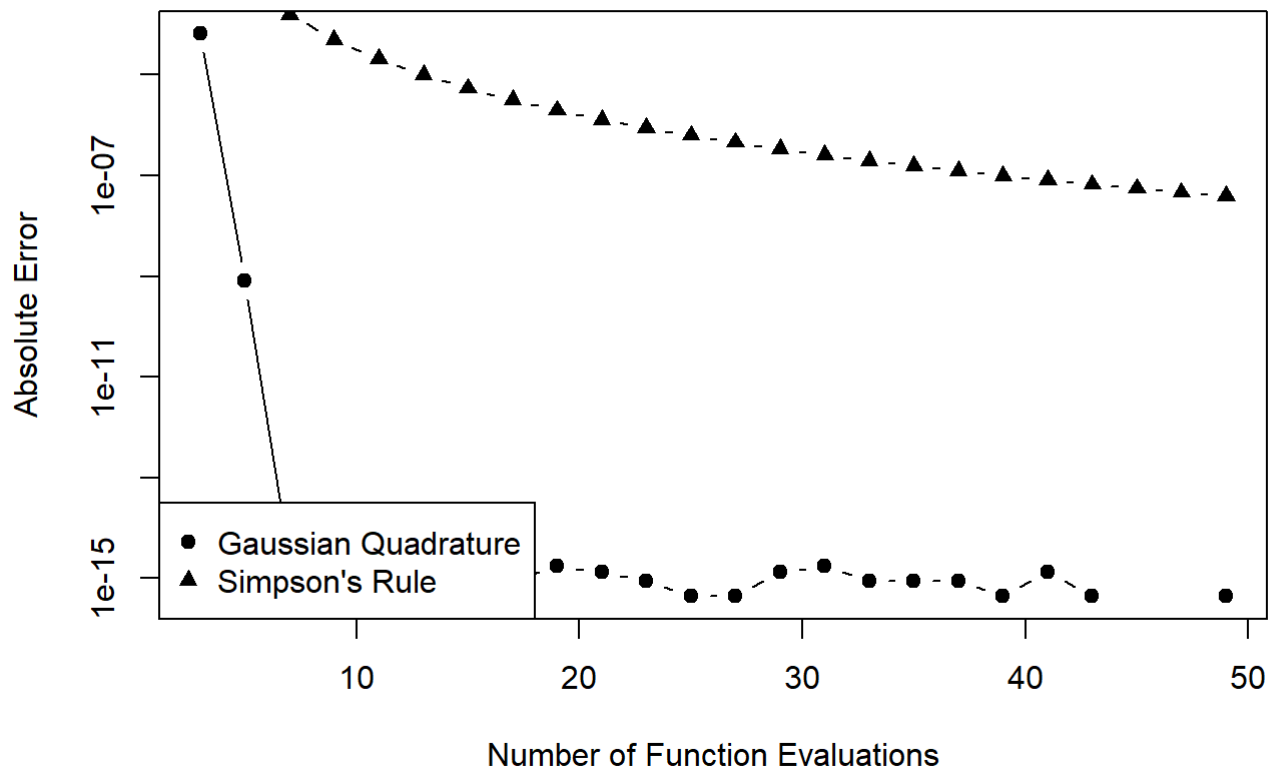
  # Legend
  legend("bottomleft",
       legend = c("Gaussian Quadrature", "Simpson's Rule"),
       pch = c(19,17))
}

```

```
plotter_compare(smooth, -1, 1, exp(1) - 1/exp(1), 50)
```

```
## Warning in xy.coords(x, y, xlabel, ylabel, log): 3 y values <= 0 omitted from
## logarithmic plot
```

Gaussian vs Simpson Convergence



The investigation reveals that Gaussian quadrature demonstrates the highest rate of convergence when applied to e^x , achieving superior precision with significantly fewer nodes than other test functions. If we recall the error of the Gaussian quadrature method,

$$\int_a^b f(x)dx - Q_n(f) = \frac{f^{(2n)}(\xi)}{(2n)!}(p_n, p_n),$$

and we note that for $f(x) = e^x$, the $2n$ th derivative is always e^x , making the numerator $f^{(2n)}(\xi)$ some constant. We also note that (p_n, p_n) is also some constant, and thus the growth of the denominator $(2n)!$ implies that the error diminishes to zero for this choice of f rapidly.

The superior performance of Gaussian quadrature compared to Simpson's rule can be understood through how each method uses function evaluations. Composite Simpson's rule improves accuracy by subdividing the interval and applying a local quadratic approximation on each sub-interval. As a result, its convergence is governed by the fourth derivative of the integrand, leading to algebraic error decay as the number of sub-intervals increases.

In contrast, Gaussian quadrature uses the same number of function evaluations in a fundamentally different way. Rather than refining the interval, it selects both the nodes and weights optimally so that the quadrature rule is exact for all polynomials up to degree $2n - 1$. This allows Gaussian quadrature to better capture the behavior of the integrand with fewer evaluations, especially for smooth functions like e^x .

7 Conclusion

In this paper, we derived the Gaussian quadrature method on the interval $[-1, 1]$ and demonstrated that, by selecting nodes as the roots of Legendre polynomials, the resulting quadrature yields the maximum degree of precision, $2n - 1$, for an n -point rule. This optimality is a result of the orthogonality properties of the Legendre polynomials and allows Gaussian quadrature to integrate high-degree polynomials exactly, given a certain number of nodes used.

I implemented the method computationally and compared its performance to the composite Simpson's rule. By measuring error as a function of the number of function evaluations, we obtained a fair comparison of computational efficiency. Numerical experiments showed that Gaussian quadrature significantly outperforms Simpson's for smooth functions, achieving much lower error with fewer function evaluations.

However, this advantage depends on the smoothness of the integrand. When functions are less regular or poorly approximated by polynomials, the benefits of Gaussian quadrature may be reduced. Despite this limitation, Gaussian quadrature remains a powerful and efficient method for numerical integration, particularly in applications involving smooth functions.

These results highlight the importance of node selection in numerical integration processes and highlight how incorporating deeper mathematical theory, like orthogonality, can lead to substantial gains in accuracy.

References

- [1] The Math Guy. *Numerical Integration - Gaussian Quadrature*. 2018. URL: <https://youtu.be/sDCyx0sU1Gc?si=pbXMp0RY6N2HGXGD>.
- [2] Anne Kvaerno and Andre Massing. *Numerical integration: Newton-Cotes and Gauß quadrature formulas [Lecture Notes]*. Course TMA4125, Norwegian University of Science and Technology. 2021.
- [3] Timothy Sauer. *Numerical Analysis*. Pearson Education, Inc, 2012.
- [4] Stoer and Bulirsch. *Introduction to Numerical Analysis*. Springer, 2002.